

DIAGNOSTIC PRATIQUE · DÉFI 1/4 · 3 MINUTES

Écris. Puis lance.

Crée `annoncerDepart()`, sans paramètre. Elle affiche « La partie commence ». Appelle-la une fois.

Nom ou code élève : _____

20 minutes au total. 3 + 4 + 5 + 6 minutes de code, puis 2 minutes pour remettre. Dans le hub, écris dans l'éditeur. Ce papier est le secours sans écran.

Écris → Exécute → Compare.

Teste un cas, observe la console, corrige une chose, puis reteste. Les indices sont facultatifs et conservés dans le suivi.

Ce que ton code doit réussir

☐ Console : La partie commence.

☐ La phrase vient de la fonction appelée, pas d'un affichage isolé.

Mon code

Mon test : attendu → obtenu

☐ Je l'explique. ☐ Je le marque « À reprendre ». Les deux permettent de conserver le travail, sans attribuer une fausse réussite.

DIAGNOSTIC PRATIQUE · DÉFI 2/4 · 4 MINUTES

Une fonction, deux prénoms.

Écris saluer(prenom). Elle affiche « Bonjour » suivi du prénom reçu. Appelle-la avec « Nora », puis « Sami ».

Ton code est à écrire. Pas de projet ni de fichier HTML à créer.

Écris → Exécute → Compare.

Teste un cas, observe la console, corrige une chose, puis reteste. Les indices sont facultatifs et conservés dans le suivi.

Ce que ton code doit réussir

- saluer("Nora") → Bonjour Nora.
- saluer("Sami") → Bonjour Sami.
- saluer("Lina") → Bonjour Lina.

Mon code

Mon test : attendu → obtenu

☐ Je l'explique. ☐ Je le marque « À reprendre ». Les deux permettent de conserver le travail, sans attribuer une fausse réussite.

DIAGNOSTIC PRATIQUE · DÉFI 3/4 · 5 MINUTES

Renvoie, puis garde.

Écris `doubler(nombre)` : elle renvoie le double. Garde le résultat de `doubler(6)` dans `scoreDouble`, puis affiche cette variable.

Ton code est à écrire. Pas de projet ni de fichier HTML à créer.

Écris → Exécute → Compare.

Teste un cas, observe la console, corrige une chose, puis reteste. Les indices sont facultatifs et conservés dans le suivi.

Ce que ton code doit réussir

- ☐ `doubler(6)` → nombre 12.
- ☐ `doubler(9)` → nombre 18 ; `doubler(0)` → 0.
- ☐ `scoreDouble` contient 12 ; `console.log` affiche cette variable.

Mon code

Mon test : attendu → obtenu

☐ Je l'explique. ☐ Je le marque « À reprendre ». Les deux permettent de conserver le travail, sans attribuer une fausse réussite.

DIAGNOSTIC PRATIQUE · DÉFI 4/4 · 6 MINUTES

Réutilise ton résultat.

Écris `additionner(pommes, poires)`, qui renvoie leur somme. Garde `additionner(2, 3)` dans `total`. Réutilise `total` avec 4 dans un second appel, stocké dans `totalSuivant`. Affiche les deux variables.

Ton code est à écrire. Pas de projet ni de fichier HTML à créer.

Écris → Exécute → Compare.

Teste un cas, observe la console, corrige une chose, puis reteste. Les indices sont facultatifs et conservés dans le suivi.

Ce que ton code doit réussir

- ☐ `additionner(2, 3) → nombre 5.`
- ☐ `additionner(4, 7) → 11 ; additionner(0, 0) → 0.`
- ☐ `total` contient 5 ; `totalSuivant` réutilise `total` et contient 9.

Mon code

Mon test : attendu → obtenu

☐ Je l'explique. ☐ Je le marque « À reprendre ». Les deux permettent de conserver le travail, sans attribuer une fausse réussite.

2 minutes de remise : vérifie les quatre programmes, clique sur « Remettre mes quatre programmes » et attends le reçu serveur. Un code incomplet peut être remis.